

Лекция 2: Написание простых программ с простейшими логическими конструкциями

Введение

На прошлой лекции мы научились создавать переменные и выводить текст на экран. Но программа, которая просто хранит данные и выводит их, не очень полезна. Настоящие программы **обрабатывают данные**: складывают числа, сравнивают значения, принимают решения.

Сегодня мы изучим инструменты, которые позволяют превратить статичный код в работающие программы:

- **Арифметические операторы** — чтобы выполнять вычисления
- **Операторы сравнения** — чтобы сравнивать данные между собой
- **Логические операторы** — чтобы комбинировать условия
- **Преобразование типов** — чтобы работать с данными разных типов

1) Арифметические операторы

Арифметические операторы в Java работают так же, как в обычной математике. Они позволяют выполнять базовые вычисления над числами. Ознакомиться с ними можно далее.

Оператор	Название	Пример	Результат
+	Сложение	5 + 3	8
-	Вычитание	5 - 3	2
*	Умножение	5 * 3	15
/	Деление	15 / 3	5
%	Остаток от деления	17 % 5	2

Таблица 1. Основные арифметические операторы Java

Особенности деления /

Оператор деления работает по-разному в зависимости от типов операндов:

```
public class DivisionExample {  
    public static void main(String[] args) {  
        // Деление целых чисел - результат целое число  
        int a = 10;  
    }  
}
```

```

int b = 3;
int result1 = a / b;
System.out.println("10 / 3 = " + result1); // Выведет: 10 / 3 = 3

// Деление с дробными числами - результат дробный
double c = 10.0;
double d = 3.0;
double result2 = c / d;
System.out.println("10.0 / 3.0 = " + result2); // Выведет: 10.0 / 3.0 = 3
.3333333333333335

// Смешанное деление - результат зависит от типов
double result3 = a / b; // a и b - int, сначала выполнится целочисленн
ое деление
System.out.println("10 / 3 (как double) = " + result3); // Выведет: 10 /
3 (как double) = 3.0

double result4 = a / d; // a - int, d - double => результатом double
System.out.println("10 / 3.0 = " + result4); // Выведет: 10 / 3.0 = 3.33
3333333333333335
    }
}

```

Важное правило: Если оба операнда целые, то и результат деления будет целым (дробная часть отбрасывается). Если хотя бы один операнд дробный, результат будет дробным.

Оператор остатка от деления %

Этот оператор очень полезен во многих задачах. Он возвращает остаток от деления первого числа на второе:

```

public class RemainderExample {
    public static void main(String[] args) {
        System.out.println("17 % 5 = " + (17 % 5)); // 2 (17 = 5*3 + 2)
        System.out.println("20 % 4 = " + (20 % 4)); // 0 (20 = 4*5 + 0)
        System.out.println("7 % 9 = " + (7 % 9)); // 7 (7 = 9*0 + 7)

        // Проверка на чётность
        int number = 42;
        if (number % 2 == 0) {
            System.out.println(number + " - чётное число");
        } else {
            System.out.println(number + " - нечётное число");
        }
    }
}

```

```

// Проверка делимости на 3
int year = 2024;
if (year % 4 == 0) {
    System.out.println(year + " - високосный год");
}
}
}

```

Где применяется %:

- Проверка чисел на чётность/нечётность
- Проверка делимости
- Получение последней цифры числа
- Работа с циклическими процессами (часы, календари)

Приоритет операций

В Java, как и в математике, операции имеют разный приоритет:

1. **Самый высокий приоритет:** умножение *, деление /, остаток %
2. **Ниже приоритет:** сложение +, вычитание -
3. **Самый низкий приоритет:** присваивание =

```

public class PriorityExample {
    public static void main(String[] args) {
        // Сначала умножение, потом сложение
        int result1 = 5 + 3 * 2;
        System.out.println("5 + 3 * 2 = " + result1); // 11, а не 16!

        // Скобки меняют приоритет
        int result2 = (5 + 3) * 2;
        System.out.println("(5 + 3) * 2 = " + result2); // 16

        // Сложный пример
        int result3 = 10 - 4 / 2 + 3 * 2;
        // 1) 4 / 2 = 2
        // 2) 3 * 2 = 6
        // 3) 10 - 2 = 8
        // 4) 8 + 6 = 14
        System.out.println("10 - 4 / 2 + 3 * 2 = " + result3); // 14

        // Лучше использовать скобки для ясности
        int result4 = 10 - (4 / 2) + (3 * 2);
        System.out.println("Со скобками: " + result4); // тоже 14
    }
}

```

```
}
```

Составные операторы присваивания

Часто встречается ситуация, когда нужно изменить значение переменной, используя её текущее значение:

```
int x = 10;  
x = x + 5; // увеличиваем x на 5
```

Для таких случаев есть сокращённая запись:

```
java  
Копировать  
Скачать  
int x = 10;  
x += 5; // то же самое, что x = x + 5
```

```
int y = 20;  
y -= 3; // y = y - 3
```

```
int z = 5;  
z *= 4; // z = z * 4
```

```
int a = 15;  
a /= 3; // a = a / 3
```

```
int b = 17;  
b %= 5; // b = b % 5
```

2) Операторы сравнения

Операторы сравнения используются для сравнения двух значений. Результатом сравнения всегда является логическое значение — **true** (истина) или **false** (ложь).

Оператор	Значение	Пример (x = 5)	Результат
==	равно	x == 5	true
!=	не равно	x != 3	true
>	больше	x > 3	true
<	меньше	x < 3	false
>=	больше или равно	x >= 5	true
<=	меньше или равно	x <= 4	false

Таблица 2. Операции сравнения в Java

```

public class ComparisonExample {
    public static void main(String[] args) {
        int age = 25;
        double temperature = 36.6;
        boolean isStudent = true;

        // Сравнение чисел
        System.out.println("age == 25: " + (age == 25)); // true
        System.out.println("age != 30: " + (age != 30)); // true
        System.out.println("age > 18: " + (age > 18)); // true
        System.out.println("age < 20: " + (age < 20)); // false
        System.out.println("age >= 25: " + (age >= 25)); // true
        System.out.println("age <= 24: " + (age <= 24)); // false

        // Сравнение дробных чисел
        System.out.println("temperature == 36.6: " + (temperature == 36.6)); //
true

        // Сравнение логических значений
        System.out.println("isStudent == true: " + (isStudent == true)); // true
        System.out.println("isStudent != false: " + (isStudent != false)); // true

        // Результат сравнения можно сохранить в переменную
        boolean isAdult = age >= 18;
        System.out.println("isAdult: " + isAdult); // true
    }
}

```

Важное предупреждение про == и =

Очень частая ошибка начинающих — путаница между оператором сравнения == и оператором присваивания =:

```

int x = 5;

// Правильно: сравнение
if (x == 5) {
    System.out.println("x равен 5");
}

// ОШИБКА! Так писать нельзя
if (x = 5) { // Ошибка компиляции!
    System.out.println("Это не работает");
}

```

Компилятор Java обнаружит эту ошибку, потому что результат присваивания — это число (5), а if ожидает логическое значение (boolean).

Сравнение строк

Со строками нужно быть особенно внимательными. Оператор == сравнивает **ссылки**, а не содержимое строк:

```
public class StringComparison {
    public static void main(String[] args) {
        String str1 = "Привет";
        String str2 = "Привет";
        String str3 = new String("Привет");

        // Сравнение через ==
        System.out.println("str1 == str2: " + (str1 == str2)); // true (особеннос
ть Java)
        System.out.println("str1 == str3: " + (str1 == str3)); // false (разные об
ъекты)

        // Правильное сравнение строк через equals
        System.out.println("str1.equals(str2): " + str1.equals(str2)); // true
        System.out.println("str1.equals(str3): " + str1.equals(str3)); // true
    }
}
```

Правило: Для сравнения строк всегда используйте метод equals(), а не ==.

3) Логические операторы

Логические операторы позволяют комбинировать несколько условий в одно сложное условие. Они работают только с логическими значениями (true/false).

Оператор	Название	Значение	Пример	Результат
&&	И (AND)	true только если оба операнда true	(5 > 3) && (2 < 4)	true
	ИЛИ (OR)	true если хотя бы один операнд true	(5 > 3) (2 > 4)	true
!	НЕ (NOT)	меняет значение на противоположное	!(5 > 3)	false

Таблица 3. Основные логические операции Java

```
public class LogicalOperators {
```

```

public static void main(String[] args) {
    int age = 20;
    boolean hasLicense = true;
    boolean hasPassport = false;

    // Оператор И (&&)
    boolean canDrive = (age >= 18) && hasLicense;
    System.out.println("Может водить: " + canDrive); // true

    // Оператор ИЛИ (||)
    boolean canFly = hasPassport || hasLicense; // для примера
    System.out.println("Может лететь: " + canFly); // true (есть license)

    // Оператор НЕ (!)
    boolean isChild = !(age >= 18);
    System.out.println("Ребёнок: " + isChild); // false

    // Сложное условие
    boolean canEnter = (age >= 21) && (hasPassport || hasLicense);
    System.out.println("Может войти: " + canEnter); // false (age < 21)
}
}

```

Таблицы истинности

Для оператора И (&&):

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

Таблица 4. Таблица истинности «И»

Для оператора ИЛИ (||):

A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

Таблица 5 Таблица истинности «ИЛИ»

Для оператора НЕ (!):

A	!A
true	false
false	true

Таблица 6. Таблица истинности «НЕ»

Приоритет логических операторов

Логические операторы тоже имеют приоритет:

1. Сначала выполняется ! (НЕ)
2. Затем && (И)
3. Затем || (ИЛИ)

```
public class LogicalPriority {
    public static void main(String[] args) {
        boolean a = true;
        boolean b = false;
        boolean c = true;

        // Пример с приоритетом
        boolean result1 = a || b && c;
        // Сначала b && c = false, затем a || false = true
        System.out.println("a || b && c = " + result1); // true

        // Со скобками - совсем другое
        boolean result2 = (a || b) && c;
        // Сначала (a || b) = true, затем true && true = true
        System.out.println("(a || b) && c = " + result2); // true

        // Ещё пример
        boolean result3 = !a && b || c;
        // !a = false, false && b = false, false || c = true
        System.out.println("!a && b || c = " + result3); // true
    }
}
```

Совет: Как и с арифметикой, используйте скобки для ясности, даже если знаете приоритет.

"Ленивые" вычисления (short-circuit evaluation)

Операторы && и || в Java работают по принципу "ленивых вычислений". Это значит, что правое выражение вычисляется только если это необходимо:

```
public class ShortCircuit {
```



```

public static void main(String[] args) {
    int x = 5;
    int y = 0;

    // Второе условие не будет вычисляться, потому что первое уже false
    if (x > 10 && (y / x > 0)) {
        System.out.println("Это не выполнится");
    }

    // Безопасно: деление на ноль не произойдёт
    if (x != 0 && (y / x > 0)) {
        System.out.println("Безопасно");
    }

    // ОПАСНО! Здесь будет деление на ноль
    // if ((y / x > 0) && x != 0) { // Ошибка!
    //     System.out.println("Опасно");
    // }
}

```

Это свойство очень полезно для защиты от ошибок: сначала проверяем, можно ли выполнить операцию, а потом её выполняем.

4) Преобразование типов

В Java строгая типизация, но иногда нужно преобразовать значение из одного типа в другой. Преобразование бывает двух видов: неявное (автоматическое) и явное (ручное).

Неявное преобразование (расширение)

Неявное преобразование происходит автоматически, когда мы присваиваем значение меньшего типа переменной большего типа. Это безопасно, потому что больший тип гарантированно вместит значение.

Цепочка расширения:

```

byte -> short -> int -> long -> float -> double
          ↑
        char

```

java

Копировать

Скачать

```

public class WideningConversion {
    public static void main(String[] args) {
        // int в double
        int age = 25;
        double ageDouble = age; // автоматическое преобразование
        System.out.println("ageDouble: " + ageDouble); // 25.0

        // byte в int
        byte small = 100;
        int bigger = small; // автоматически
        System.out.println("bigger: " + bigger); // 100

        // char в int (получаем код символа)
        char letter = 'A';
        int code = letter; // автоматически
        System.out.println("Код символа A: " + code); // 65

        // int в float
        int count = 7;
        float floatCount = count; // автоматически
        System.out.println("floatCount: " + floatCount); // 7.0

        // Важно: при преобразовании int в float может быть потеря точ
ности
        int bigInt = 123456789;
        float bigFloat = bigInt;
        System.out.println("bigInt: " + bigInt);
        System.out.println("bigFloat: " + bigFloat); // может быть округлено
    }
}

```

Когда происходит неявное преобразование:

- При присваивании
- В арифметических выражениях (если типы разные, меньший расширяется до большего)
- При передаче аргументов в методы

```

public class AutomaticInExpressions {
    public static void main(String[] args) {
        int a = 5;
        double b = 2.5;

        // a автоматически расширяется до double
        double result = a + b; // результат double
        System.out.println("result: " + result); // 7.5
    }
}

```

```

byte x = 10;
byte y = 20;

// ВНИМАНИЕ! Результат операций с byte/short - всегда int
int sum = x + y; // нужно int, не byte!
// byte sum2 = x + y; // ОШИБКА!
}
}

```

Явное преобразование (сужение)

Явное преобразование нужно, когда мы хотим поместить значение большего типа в переменную меньшего типа. Это **опасно** — можно потерять данные. Программист должен явно указать, что он понимает риски.

Синтаксис: (тип) значение

```

public class NarrowingConversion {
    public static void main(String[] args) {
        // double в int - теряем дробную часть
        double price = 99.99;
        int intPrice = (int) price; // явное преобразование
        System.out.println("price: " + price);
        System.out.println("intPrice: " + intPrice); // 99 (дробная часть отброшена)

        // long в int - риск переполнения
        long bigNumber = 3_000_000_000L; // 3 миллиарда
        int intNumber = (int) bigNumber;
        System.out.println("bigNumber: " + bigNumber);
        System.out.println("intNumber: " + intNumber); // результат непредсказуем!

        // int в byte - потеря старших байтов
        int large = 1000;
        byte small = (byte) large;
        System.out.println("large: " + large);
        System.out.println("small: " + small); // результат -24 из-за переполнения

        // double в float
        double precise = 3.141592653589793;
        float lessPrecise = (float) precise;
        System.out.println("precise: " + precise);
    }
}

```

a

```
System.out.println("lessPrecise: " + lessPrecise); // точность снижен
```

```
// int в char
```

```
int code = 65;
```

```
char symbol = (char) code;
```

```
System.out.println("Символ с кодом 65: " + symbol); // 'A'
```

```
}
```

```
}
```

Преобразование при арифметических операциях

В Java действуют следующие правила при вычислении выражений:

1. Если один из операндов double, всё выражение становится double
2. Иначе, если один из операндов float, всё становится float
3. Иначе, если один из операндов long, всё становится long
4. Иначе все byte, short, char преобразуются в int

java

Копировать

Скачать

```
public class TypePromotion {
```

```
    public static void main(String[] args) {
```

```
        byte b1 = 10;
```

```
        byte b2 = 20;
```

```
// byte + byte = int
```

```
int sum1 = b1 + b2; // нужно int
```

```
// byte sum2 = b1 + b2; // ОШИБКА!
```

```
short s1 = 100;
```

```
short s2 = 200;
```

```
// short + short = int
```

```
int sum3 = s1 + s2; // нужно int
```

```
// Смешанные типы
```

```
int i = 5;
```

```
long l = 10L;
```

```
float f = 2.5f;
```

```
double d = 3.0;
```

```
// i + l = long
```

```
long r1 = i + l;
```

```

// l + f = float
float r2 = l + f;

// f + d = double
double r3 = f + d;

// Всё вместе: i + l + f + d = double
double r4 = i + l + f + d;
}
}

```

Практические примеры преобразований

```

public class ConversionPractice {
    public static void main(String[] args) {
        // Пример 1: Вычисление среднего арифметического
        int sum = 15;
        int count = 4;

        // Неправильно: целочисленное деление
        int badAverage = sum / count;
        System.out.println("Неправильное среднее: " + badAverage); // 3

        // Правильно: преобразуем к double
        double goodAverage = (double) sum / count;
        System.out.println("Правильное среднее: " + goodAverage); // 3.75

        // Пример 2: Получение цифр числа
        int number = 123;
        int lastDigit = number % 10; // 3
        int firstDigit = number / 100; // 1

        System.out.println("Число: " + number);
        System.out.println("Последняя цифра: " + lastDigit);
        System.out.println("Первая цифра: " + firstDigit);

        // Пример 3: Округление
        double value = 7.8;
        int rounded = (int) (value + 0.5); // простой способ округления
        System.out.println("Округление " + value + " -> " + rounded); // 8
    }
}

```

5) Практические примеры программ

Теперь, когда мы изучили все операторы, давайте напишем несколько полноценных программ.

Пример 1: Калькулятор скидки

```
public class DiscountCalculator {
    public static void main(String[] args) {
        double price = 1500.0;
        int age = 25;
        boolean isStudent = true;

        double discount = 0.0;

        // Скидка 10% для студентов
        if (isStudent) {
            discount = discount + 0.1;
        }

        // Скидка 5% для людей старше 60
        if (age >= 60) {
            discount = discount + 0.05;
        }

        // Скидка 15% для именинников (допустим, сегодня день рождения)
        boolean isBirthday = true;
        if (isBirthday) {
            discount = discount + 0.15;
        }

        // Максимальная скидка не может превышать 30%
        if (discount > 0.3) {
            discount = 0.3;
        }

        double finalPrice = price * (1 - discount);

        System.out.println("Исходная цена: " + price + " руб.");
        System.out.println("Скидка: " + (discount * 100) + "%");
        System.out.println("Итоговая цена: " + finalPrice + " руб.");
    }
}
```

Пример 2: Проверка пароля

```

public class PasswordChecker {
    public static void main(String[] args) {
        String correctPassword = "Java2024";
        String userInput = "java2024"; // представим, что ввёл пользователь

        boolean isCorrect = userInput.equals(correctPassword);
        boolean isCorrectIgnoreCase = userInput.equalsIgnoreCase(correctPass
word);

        System.out.println("Пароль введён: " + userInput);
        System.out.println("Пароль верный (с учётом регистра): " + isCorrec
t);

        System.out.println("Пароль верный (без учёта регистра): " + isCorrec
tIgnoreCase);

        // Проверка длины пароля
        boolean isValidLength = userInput.length() >= 8;
        System.out.println("Длина пароля достаточная: " + isValidLength);
    }
}

```

Пример 3: Определение времени года

```

public class SeasonDeterminer {
    public static void main(String[] args) {
        int month = 7; // июль

        boolean isWinter = month == 12 || month == 1 || month == 2;
        boolean isSpring = month >= 3 && month <= 5;
        boolean isSummer = month >= 6 && month <= 8;
        boolean isAutumn = month >= 9 && month <= 11;

        System.out.print("Месяц " + month + " - это ");

        if (isWinter) {
            System.out.println("зима");
        } else if (isSpring) {
            System.out.println("весна");
        } else if (isSummer) {
            System.out.println("лето");
        } else if (isAutumn) {
            System.out.println("осень");
        } else {
            System.out.println("некорректный месяц");
        }
    }
}

```

```
}  
}
```

Пример 4: Калькулятор индекса массы тела

```
public class BMICalculator {  
    public static void main(String[] args) {  
        double weight = 75.5; // вес в кг  
        double height = 1.78; // рост в метрах  
  
        double bmi = weight / (height * height);  
  
        System.out.println("Вес: " + weight + " кг");  
        System.out.println("Рост: " + height + " м");  
        System.out.println("ИМТ: " + bmi);  
  
        boolean isUnderweight = bmi < 18.5;  
        boolean isNormal = bmi >= 18.5 && bmi < 25;  
        boolean isOverweight = bmi >= 25 && bmi < 30;  
        boolean isObese = bmi >= 30;  
  
        System.out.print("Категория: ");  
        if (isUnderweight) {  
            System.out.println("Недостаточный вес");  
        } else if (isNormal) {  
            System.out.println("Нормальный вес");  
        } else if (isOverweight) {  
            System.out.println("Избыточный вес");  
        } else if (isObese) {  
            System.out.println("Ожирение");  
        }  
    }  
}
```

6) Типичные ошибки и как их избежать

Ошибка 1: Путаница между = и ==

```
// НЕПРАВИЛЬНО  
int x = 5;  
if (x = 10) { // Ошибка компиляции  
    System.out.println("x равен 10");  
}
```


// ПРАВИЛЬНО

```
int x = 5;
if (x == 10) {
    System.out.println("x равен 10");
}
```

Ошибка 2: Сравнение строк через ==

// НЕПРАВИЛЬНО

```
String s1 = "Hello";
String s2 = new String("Hello");
if (s1 == s2) { // false, хотя строки одинаковые
    System.out.println("Строки равны");
}
```

// ПРАВИЛЬНО

```
if (s1.equals(s2)) {
    System.out.println("Строки равны");
}
```

Ошибка 3: Деление целых чисел

java

Копировать

Скачать

// НЕПРАВИЛЬНО

```
int a = 5;
int b = 2;
double result = a / b; // result = 2.0, а не 2.5
```

// ПРАВИЛЬНО

```
double result = (double) a / b; // 2.5
```

Ошибка 4: Забыли про приоритет операций

// НЕПРАВИЛЬНО

```
int x = 5 + 3 * 2; // x = 11, а ожидалось 16
```

// ПРАВИЛЬНО (если нужно сложить сначала)

```
int x = (5 + 3) * 2; // 16
```

Ошибка 5: Проверка на чётность

// НЕПРАВИЛЬНО (иногда так ошибаются)

```
int num = 7;
if (num % 2 = 0) { // Ошибка: присваивание вместо сравнения
    System.out.println("Чётное");
}

// ПРАВИЛЬНО
if (num % 2 == 0) {
    System.out.println("Чётное");
} else {
    System.out.println("Нечётное");
}
```

Итоги второй лекции

Сегодня мы изучили фундаментальные операторы Java:

Арифметические операторы:

- +, -, *, /, % для выполнения вычислений
- Приоритет: сначала *, /, %, потом +, -
- Скобки () меняют приоритет

Операторы сравнения:

- ==, !=, <, >, <=, >=
- Всегда возвращают boolean (true или false)

Логические операторы:

- && (И) — true только если оба true
- || (ИЛИ) — true если хотя бы один true
- ! (НЕ) — инвертирует значение

Преобразование типов:

- Неявное (расширение) — безопасно, происходит автоматически
- Явное (сужение) — требует указания типа в скобках, возможна потеря данных

Ключевые выводы:

1. В Java строгая типизация — всегда следите за типами данных
2. Приоритет операций важен — используйте скобки для ясности
3. % (остаток от деления) — очень полезный оператор
4. Сравнивайте строки через equals(), а не через ==
5. При делении помните про целочисленное деление